EventVLA: Event-Driven Visual Evidence Memory for Long-Horizon Vision-Language-Action Policies

Ganlin Yang[1,2*], **Zhangzheng Tu**[4,3*], **Yuqiang Yang**[2*], **Sitong Mao**[5], **Junyi Dong**[5]
Tianxing Chen[6], **Jiaqi Peng**[7,2], **Jing Xiong**[8,2], **Jiafei Cao**[2], **Jifeng Dai**[7], **Wengang Zhou**[1]
Yao Mu[3,2,6†], **Tai Wang**[2†]

[1]University of Science and Technology of China [2]Shanghai AI Laboratory
[3]Shanghai Jiao Tong University [4]Dalian University of Technology [5]Huawei Technologies Co., Ltd.
[6]The University of Hong Kong [7]Tsinghua University [8]Peking University

Project Page: EventVLA

Abstract: Memory remains a critical bottleneck for long-horizon robotic manipulation, as standard Vision-Language-Action (VLA) policies often fail when task-relevant cues become occluded or unobservable over time. While existing memory-augmented methods utilize historical context, they either suffer from severe information bottlenecks, incur high latency via decoupled dual systems, or rely on unselective buffers that accumulate massive visual redundancies. To address these limitations, we introduce **EventVLA**, an end-to-end framework founded on the concept of **sparse visual evidence memory** that comprises two core components: foundational *visual anchors* to retain initial and short-term contexts, and a dynamic *Keyframe Evidence Memory (KEM)* module. Specifically, KEM directly predicts future keyframe probabilities from the VLA's latent embeddings to autonomously capture and store sparse, task-critical visual events. This foresight-driven mechanism empowers the policy to dynamically evaluate the future causal utility of current observations, preserving transient visual evidence before it becomes unobservable. Furthermore, we propose **RoboTwin-MeM**, a diagnostic benchmark specifically designed to evaluate non-Markovian manipulation tasks with interactive visual evidence. Extensive evaluations show that across 17 memory-requiring simulation tasks and 4 real-world bimanual tasks, EventVLA achieves an average success rate improvement of +40% over state-of-the-art memory-augmented VLAs. The code, models and datasets are available at `https://github.com/InternRobotics/EventVLA`.

Keywords: Memory, Robotic Manipulation, Robotic Benchmark

1 Introduction

Recent Vision-Language-Action (VLA) policies excel in generalizable and fine-grained manipulation [1, 2, 3, 4, 5], yet they predominantly operate under a strict Markovian assumption. This implicitly assumes all task-relevant information remains persistently visible. In reality, physical workspaces change dynamically, and agents must constantly retain intermediate states, such as the original location of a displaced item to guide subsequent actions. To address this non-Markovian challenge, memory-aware VLAs have emerged across three paradigms [6]. First, Dual-system Memory-VLAs [7, 8, 9] decouple cognition from control but suffer from high latency and severe error propagation. Second, Recurrent architectures [10, 11] compress history into hidden states, creating an information bottleneck that discards fine-grained visual details. Third, Memory Buffers [12, 13] preserve visual fidelity but blindly accumulate redundant frames without a selective mechanism.

*Equal contribution. †Corresponding authors.

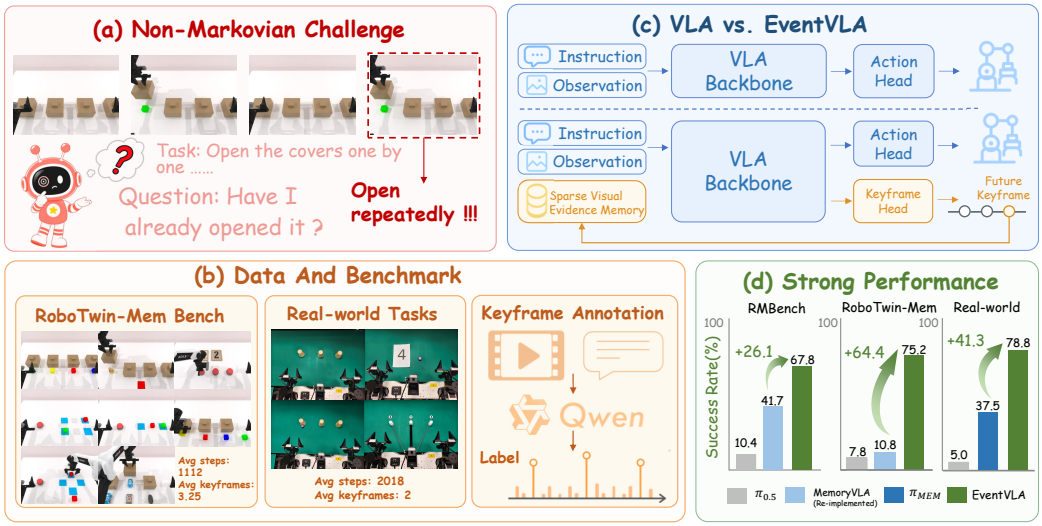


Figure 1: **Overview of EventVLA.** EventVLA tackles long-horizon, memory-requiring manipulation tasks by storing sparse, task-critical visual evidence. The figure illustrates the (a) non-Markovian challenge, (b) our proposed and evaluated benchmarks, (c) event-driven memory design, and (d) strong gains across simulation and real-world tasks.

Consequently, a critical question remains: exactly *when* and *what* visual evidence should a VLA preserve to maximize execution success without overwhelming computational limits?

To avoid the massive redundancy of standard memory buffers [12, 14], we identify that a sparse set of historical keyframes provides sufficient context for many long-horizon tasks. We define these as foundational **visual anchors**: the initial frame (capturing the invariant global layout) and a short-term history window (providing local motion cues). While these heuristic anchors efficiently solve structurally simple memory-requiring tasks, they fundamentally fail in complex interactive scenarios where task-critical evidence emerges unexpectedly and subsequently disappears. For example, a robot may briefly observe an object's color when lifting an opaque cover, or need to track a designated target that later becomes occluded. Such transient visual evidence cannot be recovered from initial or recent frames; it manifests as a interactive *sparse event* that must be actively captured and preserved.

Building upon this insight, we introduce **EventVLA**, an end-to-end framework rooted in sparse visual evidence memory. EventVLA eliminates historical redundancy by seamlessly combining foundational visual anchors with a dynamic **Keyframe Evidence Memory (KEM)** module. Unlike rigid, rule-based heuristics, KEM establishes an autonomous, data-driven mechanism designed to actively capture transient, interaction-driven events. Specifically, by performing foresight-driven keyframe predictions over the upcoming execution horizon, KEM empowers the VLA policy to proactively schedule sparse memory writes for critical intermediate states. This predictive strategy ensures that transient visual evidence is captured long before it becomes explicitly required by the task, seamlessly bridging the temporal gap between its brief appearance and its eventual use in downstream execution. To learn this capability without prohibitive manual annotation, we develop an offline, Qwen3-VL-based [15] automatic labeling pipeline that extracts precise keyframe supervision from demonstrations.

Beyond algorithm design, evaluating memory-augmented policies requires benchmarks that accurately capture the non-Markovian dynamics in real-world manipulation, where task-critical evidence often manifests only transiently during intermediate interactions. Because existing benchmarks like RMBench [8] can largely be solved by basic visual anchors alone, we introduce **RoboTwin-MeM**,

a diagnostic simulation benchmark explicitly featuring such genuinely non-Markovian scenarios. It comprises 8 challenging tasks, where the required intermediate keyframes systematically scale from 1 to 5. Extensive evaluations demonstrate EventVLA's superiority across diverse domains. It sets a new state-of-the-art on conventional memory-oriented tasks (67.8% on RMBench) and achieves a 75.2% average success rate on the newly transient-memory-required RoboTwin-MeM, vastly outperforming existing memory-based VLAs. Furthermore, in demanding real-world bimanual tasks, EventVLA significantly surpasses both reactive ($\pi_{0.5}$ [1]) and memory-augmented (π_{MEM} [9]) baselines with up to 80% success rates, confirming its robust non-Markovian situational awareness.

2 Related Work

2.1 Memory-Augmented Policies for Long-Horizon Manipulation

Recent Vision-Language-Action (VLA) foundation models [1, 2, 16, 17, 5, 3, 4, 18, 19, 20, 21, 22, 23, 24, 25] achieve remarkable generalizability but are fundamentally memoryless. Operating under a strict Markovian assumption, they struggle with non-Markovian tasks where critical visual information is transient or occluded. To address this, memory-augmented VLAs have emerged across three paradigms. First, *dual-system Memory-VLAs* [7, 8, 9, 26, 27] use a high-level VLM for planning but suffer from error propagation and high inference latency. Second, *recurrent memory architectures* [10, 11, 28, 29, 30] compress histories into hidden states, creating an information bottleneck that discards fine-grained visual details. Third, *Memory Buffers* [31, 12, 13, 32, 33, 34, 14] retain historical frames to bypass compression; however, existing methods blindly accumulate redundant frames, drowning out sparse key evidence and incurring heavy overhead. EventVLA optimizes this paradigm by preserving only sparse visual evidence. By combining static *visual anchors* with a dynamic *Keyframe Evidence Memory (KEM)*, EventVLA selectively captures transient states, balancing robust task execution with real-time computational efficiency.

2.2 Memory-Oriented Manipulation Benchmarks

Standard simulation suites [35, 36, 37, 38, 39] emphasize long-horizon execution rather than explicit memory reasoning, as task-relevant information typically remains persistently visible. While recent memory-centric benchmarks [40, 41, 42, 43, 44] attempt to address this, they are often limited in scale, tailored exclusively for reinforcement learning, or still feature observable states. The closest suites to ours, RMBench [8] and RoboMME [6], systematically stratify memory demands but can largely be solved by static visual anchors alone, leaving strictly non-Markovian intermediate states under-explored. To bridge this methodological gap, we introduce **RoboTwin-MeM**. Distinct from existing benchmarks, RoboTwin-MeM isolates genuinely non-Markovian manipulation tasks where critical visual evidence transiently emerges during interaction and subsequently disappears, providing a rigorous diagnostic platform to evaluate a VLA policy's capacity for intermediate state retention.

3 EventVLA Framework

3.1 Problem Formulation and Foundational Visual Anchors

We formalize long-horizon robotic manipulation as a non-Markovian decision process. Standard reactive VLA policies map the current observation o_t and language instruction l directly to an action, i.e., $a_t = \pi(o_t, l)$, which fundamentally fails when critical information becomes occluded or unobservable over time. To address this, EventVLA incorporates an explicit, external sparse visual evidence memory buffer M_t to condition action generation along with the immediate observation:

$$a_t = \pi(o_t, M_{t-1}, l) \tag{1}$$

where M_{t-1} selectively stores key historical frames to preserve essential visual evidence while minimizing informational and computational redundancy.

The memory buffer is structured as $M_t = A_t \cup E_t$, seamlessly uniting foundational *visual anchors* A_t and interaction-driven event keyframes E_t. The visual anchors A_t represent a deterministic, rule-based baseline designed to capture the permanent scene layout and immediate temporal context. Specifically, the visual anchors at timestep t consist of the initial workspace configuration o_0 and a short-term history sliding window of size K:

$$A_t = o_0 \cup o_{t-K}, \ldots, o_{t-1} \tag{2}$$

Here, the initial frame o_0 serves as a permanent spatial anchor, allowing the VLA model to preserve an invariant memory of the original scene arrangement before any displacements occur. Meanwhile, the short-term history o_{t-i} supplies the model with critical motion and task progression cues, enabling smooth and continuous action generation. However, since these rigid anchors cannot capture unpredictable, transient evidence arising midway through complex interactions, they are dynamically augmented by E_t produced by the Keyframe Evidence Memory (KEM) module, as detailed in Section 3.2 and the overall framework is shown in Fig. 2.

3.2 Keyframe Evidence Memory (KEM) Module

To actively capture transient, interaction-driven events that foundational visual anchors inherently miss, such as the brief exposure of an occluded object, we introduce the Keyframe Evidence Memory (KEM) module. To implement this mechanism efficiently, KEM is designed as a lightweight, parallel prediction head operating directly alongside the primary action heads. Rather than utilizing isolated features, the keyframe prediction head ingests the exact hidden states $h_t \in \mathbb{R}^{H \times d}$ extracted from the final layer of the VLA's autoregressive transformer, for action horizon H. Because h_t naturally encapsulates the joint embedding of visual observations and action-conditioned query tokens, the keyframe head inherits a proactive awareness of the model's future execution plan. Specifically, the keyframe head projects these shared hidden states h_t to a vector of keyframe probabilities $\hat{\mathbf{p}}_t$ spanning the future chunk horizon H:

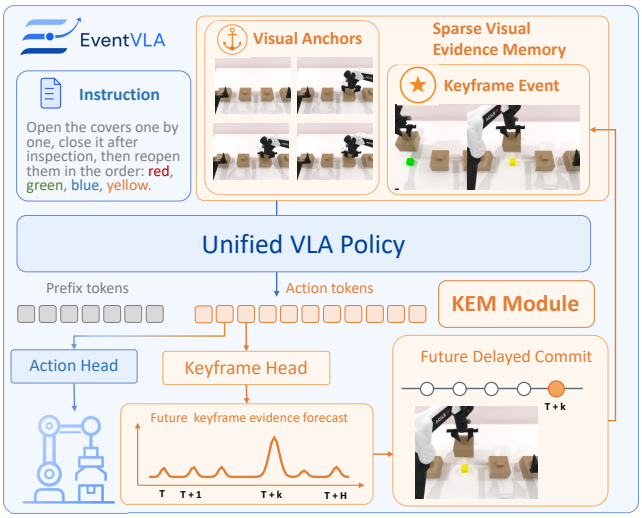


Figure 2: **EventVLA framework.** EventVLA maintains a sparse visual evidence memory composed of foundational visual anchors and interaction-driven event keyframes, and uses the KEM module to proactively commit task-critical future key observations into memory.

$$\hat{\mathbf{p}}_t = \sigma(\text{KEM}_{\text{mlp}}(h_t)) = [\hat{p}_t^1, \hat{p}_t^2, \ldots, \hat{p}_t^H]^T \in [0,1]^H \tag{3}$$

where $\sigma(\cdot)$ denotes the element-wise sigmoid function, and each scalar $\hat{p}_t^i \in [0,1]$ explicitly represents the predicted probability of the i-th future execution step being a task-critical keyframe. The rationale for this chunk-wise prediction is straightforward: a purely step-wise classifier would completely miss task-critical events that transiently manifest and vanish midway through the execution window (e.g., at step $t+i$ where $0 < i < H$). This limitation motivates KEM to adopt a foresight-driven, chunk-wise paradigm $\hat{\mathbf{p}}_t$, empowering the VLA policy to proactively map out a "memory schedule" across the entire upcoming execution horizon.

Driven by this predictive vector $\hat{\mathbf{p}}_t$, EventVLA triggers a sparse memory write *event* whenever a predicted probability crosses a threshold ($\hat{p}_t^i \geq \tau_{\text{commit}}$), dynamically committing the raw image

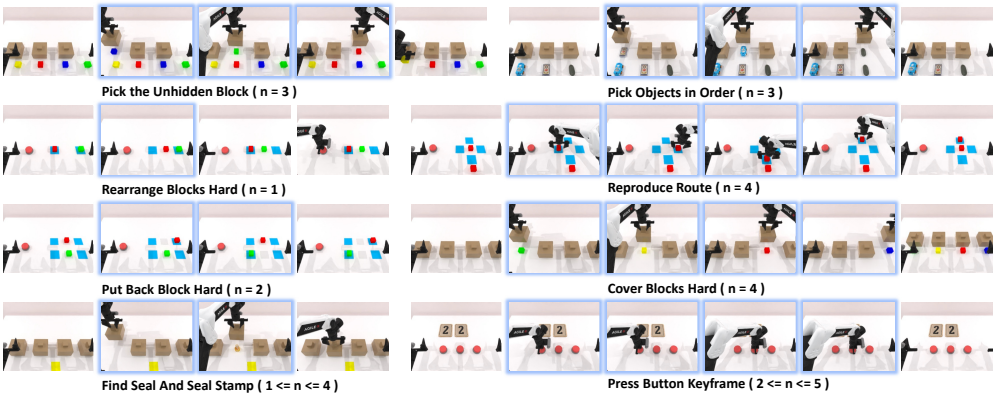


Figure 3: **Overview of the 8 evaluation tasks in the RoboTwin-MeM benchmark.** To rigorously evaluate the capacity for intermediate visual evidence retention, each task is explicitly parameterized by n (ranges from 1 to 5), denoting the exact number of transient, interaction-driven keyframes that must be memorized to succeed. These task-critical intermediate events are highlighted with blue borders.

at $t + i$ to the event buffer E_t. To satisfy real-time constraints, E_t is bounded by a maximum capacity N_{max}, managed via a First-In-First-Out (FIFO) eviction policy. At any execution step t, these dynamically accumulated event keyframes E_{t-1} are seamlessly combined with the foundational visual anchors A_t and the immediate observation o_t into a single, temporally ordered sequence:

$$I_{\text{input}} = concatenate([A_t, E_{t-1}, o_t]) \tag{4}$$

Feeding this unified sequence directly into the VLM's vision encoder allows the self-attention layers to dynamically extract complex temporal correlations across sparse historical frames, natively endowing the model with robust situational awareness for long-horizon manipulation.

3.3 End-to-End Training and Inference Details

To train the KEM module without prohibitive manual annotation costs, we employ an offline Qwen3-VL [15] automated pipeline to extract ground-truth timestamps of task-critical events. To mitigate the inherent temporal ambiguity of physical interactions, we supervise chunk-wise keyframe predictions using temporally smoothed soft labels via a sequence-averaged BCE objective (L_{kem}). The framework is optimized end-to-end alongside the standard action generation loss (L_{action}):

$$L = L_{\text{action}} + \lambda L_{\text{kem}} \tag{5}$$

To bridge the train-test distribution shift while maintaining early training stability, we apply a scheduled teacher-to-student curriculum that gradually transitions memory construction from ground-truth to autonomous predictions. During online inference, continuous keyframe probabilities naturally cluster around unfolding semantic events. To enforce strict memory sparsity and prevent redundant buffer flooding, we distill these dense predictions into discrete write events using a 1D Non-Maximum Suppression (NMS) and temporal cooldown pipeline. Comprehensive mathematical formulations regarding the soft labels, curriculum, NMS algorithm, and the automated labeling pipeline are deferred to Appendix A. Additionally, complete network structures and training configurations are detailed in Appendix B.3.

4 RoboTwin-MeM Benchmark

To systematically evaluate the capability of VLA policies to capture and retain transient visual evidence, we introduce RoboTwin-MeM, a diagnostic simulation benchmark. Developed within the RoboTwin 2.0 [35] simulation platform and built on top of the SAPIEN [45] physics engine,

RoboTwin-MeM supports both automated data synthesis and integrated policy evaluation within a unified pipeline. This infrastructure ensures scalable data generation alongside consistent, reproducible benchmarking for robotic manipulation. Furthermore, we provide fine-grained language annotations that align strictly with each action-observation pair. These annotations assign explicit linguistic descriptions to low-level interactions and state transitions, offering structured and dense supervision signals that are highly beneficial for training downstream memory modules.

The core distinction between RoboTwin-MeM and existing memory-centric suites is its explicit isolation and quantification of intermediate memory demands. While previous benchmarks often permit policies to succeed by relying merely on initial static anchors or short-term histories, RoboTwin-MeM forces the model to actively memorize unpredictable visual evidence generated midway through execution. To rigorously diagnose this capability, we explicitly parameterize task complexity using n: the exact number of intermediate event keyframes that must be dynamically preserved. As illustrated in Fig. 3, RoboTwin-MeM comprises 8 genuinely non-Markovian tasks featuring extremely long execution horizons, averaging between 430 and 1544 steps per episode. Across the benchmark, the required intermediate keyframe count n systematically ranges from 1 to 5, establishing a tiered difficulty hierarchy for non-Markovian control. The detailed task statistics and language instructions can be found in Table 4.

Crucially, this n-parameterized design allows RoboTwin-MeM to evaluate a diverse spectrum of memory capabilities beyond trivial history concatenation. First, tasks like *Pick the Unhidden Block* ($n = 3$) and *Cover Blocks Hard* ($n = 4$) demand *transient memory*; essential visual evidence is briefly exposed when a cover is lifted and completely disappears once closed, requiring the policy to instantly anchor this fleeting information. Second, the benchmark evaluates sequence tracking and counting logic via tasks like *Press Button Keyframe* ($n \in [2, 5]$), where each button press represents an execution-critical event that must be sequentially registered to dictate task success. Finally, the *Reproduce Route* task ($n = 4$) tests the model's *in-context learning* capacity, requiring the agent to observe a demonstration, extract randomized spatial keypoints, and leverage these cues in-context to duplicate the route. This coverage of transient recognition, event counting, and in-context imitation makes RoboTwin-MeM a rigorous benchmark for evaluating memory-augmented robotic policies.

5 Experiments

5.1 Performance on Simulation Benchmarks

To thoroughly assess the efficacy of EventVLA, we benchmark our framework against a comprehensive suite of state-of-the-art baselines, categorized into two major paradigms. For standard, reactive (non-memory-based) VLA policies, we select DP [3], ACT [5], $\pi_{0.5}$ [1], X-VLA [17], and QwenOFT [46]. For memory-augmented methods, we evaluate dual-system architectures, including MemER [7] and Mem-0 [8], as well as the end-to-end MemoryVLA [12] framework, where we reproduce its variants based on both the official OpenVLA-OFT [47] and QwenOFT [46] implementations. Our proposed EventVLA is also constructed upon the identical open-source QwenOFT backbone as its foundational base model.

Table 1: Overall average success rates on RMBench. Detailed per-task breakdowns are in Appendix Table 8.

Methods	Average (%)
Non Memory-based VLAs	
DP	5.8
ACT	5.9
$\pi_{0.5}$	10.4
X-VLA	9.8
QwenOFT	5.6
Dual-system Memory-VLAs	
MemER	8.7
Mem-0	42.0
End-to-end Memory-VLAs	
MemoryVLA (OpenVLA)	19.4
MemoryVLA (QwenOFT)	41.7
EventVLA& Ablations	
EventVLA (w/o initial)	33.7
EventVLA (w/o short-term)	23.8
EventVLA (VA only)	**67.8**

Evaluation on RMBench and the Efficacy of Visual Anchors: First, we evaluate our method on RMBench [8], as shown in Table 1. Because tasks in this suite primarily rely on persistent spatial layouts and fixed motion style rather than hidden intermediate states, we deploy a streamlined version of EventVLA utilizing solely foundational visual anchors. Experimental results demonstrate that this configuration achieves an average success rate of 67.8%, securing state-of-the-art performance and proving that rule-based anchors provide sufficient context

Table 2: RoboTwin-MeM benchmark results. (**Bold** : best; Underlined: second-best).

Tasks	n=1	n=2	n=3		n=4			n=5	*Total average*
	Rearrange Blocks Hard	Put Back Block Hard	Pick Objects in Order	Pick the Unhidden Block	Cover Blocks Hard	Find Seal Stamp	Reproduce Route	Press Button Keyframe	
▼ *Non Memory-based Vision-language-action Models:*									
$\pi_{0.5}$	20%	19%	1%	14%	0%	8%	0%	0%	7.8%
QwenOFT	3%	26%	0%	0%	0%	0%	0%	1%	3.8%
▼ *Dual-system Memory-based Vision-language-action Models:*									
MemER	32%	4%	12%	2%	0%	26%	3%	5%	10.5%
Mem-0	0%	0%	0%	0%	0%	0%	0%	0%	0.0%
▼ *End-to-end Memory-based Vision-language-action Models:*									
MemoryVLA (OpenVLA)	12%	0%	0%	1%	0%	10%	2%	14%	4.9%
MemoryVLA (QwenOFT)	39%	0%	1%	9%	1%	11%	0%	25%	10.8%
EventVLA (VA only)	**62%**	13%	5%	20%	0%	26%	0%	18%	18.0%
EventVLA (VA+KEM)	**62%**	**93%**	**90%**	**54%**	**94%**	**63%**	**98%**	**48%**	**75.2%**
EventVLA (implicit memory bank)	51%	9%	16%	37%	1%	68%	2%	15%	24.9%
EventVLA (hard label)	59%	77%	28%	62%	85%	36%	6%	37%	48.8%
EventVLA (w/o NMS)	62%	93%	49%	36%	10%	35%	97%	45%	53.4%
EventVLA ($N_{\max} = 2$)	51%	35%	28%	33%	39%	53%	0%	17%	32.0%
EventVLA (chunk size=30)	22%	98%	18%	28%	16%	29%	0%	38%	31.1%
EventVLA (chunk size=15)	16%	30%	2%	17%	6%	16%	10%	12%	13.6%

for simple memory-required long-horizon manipulation. To validate the structural necessity of these components, we conduct two ablation studies. Removing the initial frame (EventVLA w/o initial) or discarding the short-term history (EventVLA w/o short-term) causes the overall success rate to plummet to 33.7% and 23.8%, respectively. This confirms that both the initial global spatial reference and the short-term motion cues are indispensable for effective visual anchoring.

Evaluation on RoboTwin-MeM and the Necessity of KEM: While foundational visual anchors excel on RMBench, their limitations become starkly apparent when evaluated on RoboTwin-MeM, our diagnostic suite explicitly designed to test intermediate state memory. As detailed in Table 2, relying solely on rule-driven visual anchors (VA only) yields a mere 18.0% average success rate. This sharp drop indicates that fixed historical windows are fundamentally inadequate for tasks requiring VLA policies to retain transient visual evidence generated mid-execution. To overcome this non-Markovian bottleneck, the full EventVLA framework augments these visual anchors with the dynamic Keyframe Evidence Memory (KEM) module (VA+KEM). Experimental results reveal a qualitative leap: the complete EventVLA achieves a 75.2% success rate, outperforming all baseline models by a substantial margin. This striking performance delta (from 18.0% to 75.2%) compellingly demonstrates that KEM's dynamic event capture and foresight-driven writing mechanisms are indispensable for solving complex, long-horizon tasks that hinge on transient intermediate memory.

Evaluation on Standard Markovian Benchmarks: To verify that EventVLA preserves fundamental reactive control, we evaluate it on standard Markovian tasks in RoboTwin-2.0 [35] (Table 3). Rather than degrading performance, our memory mechanism slightly improves success rates over the memoryless QwenOFT baseline (80.0% to 83.8% on Easy; 78.0% to 81.6% on Hard), seamlessly complementing standard closed-loop execution.

Table 3: **RoboTwin2.0 benchmark results.** EventVLA outperforms its baseline foundation model QwenOFT on Markovian tasks.

Tasks	π_0	$\pi_{0.5}$	X-VLA	QwenFast	QwenOFT	EventVLA
Easy	65.9%	82.7%	72.8%	72.5%	80.0%	83.8%
Hard	58.4%	76.8%	72.8%	83.2%	78.0%	81.6%

5.2 Ablation Analysis of EventVLA

To systematically validate the structural design of the Keyframe Evidence Memory (KEM) module, we conduct ablation studies on the challenging RoboTwin-MeM suite (Table 2, bottom in gray).

Core Mechanisms: We observe that replacing explicit raw image concatenation with an implicit latent memory bank drastically drops the success rate from 75.2% to 24.9%, creating a severe information bottleneck. Similarly, substituting temporally smoothed soft labels with rigid binary targets destabilizes the predictive head, reducing performance to 48.8%.

Buffer and Horizon Management: Removing the NMS post-processing or restricting the buffer capacity ($N_{\max} = 2$) leads to redundant frame flooding and premature FIFO eviction of critical early

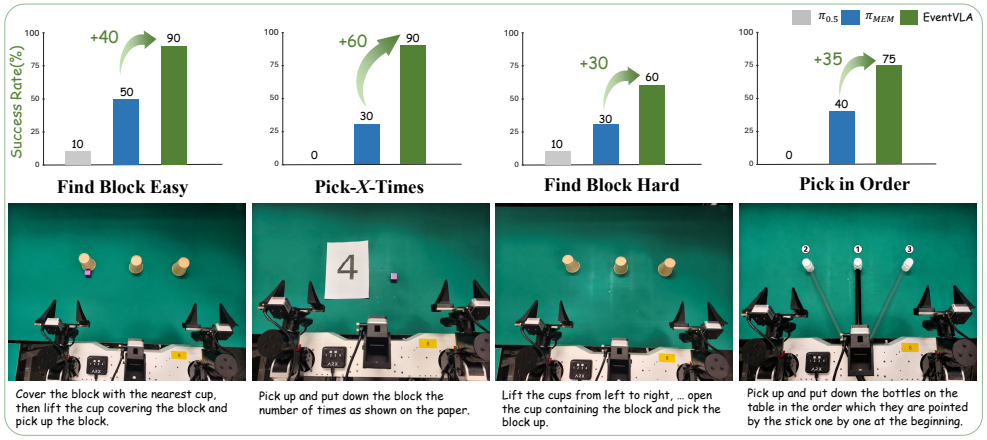


Figure 4: Real-world experimental setups and results on the ARX ACONE bimanual robot. We evaluate four memory-intensive manipulation tasks: *Find Block Easy*, *Find Block Hard*, *Pick-X-Times*, and *Pick in Order*.

evidence, degrading success rates to 53.4% and 32.0%, respectively. Finally, shrinking the execution chunk size (from 50 to 30 or 15) severely truncates KEM's foresight window, preventing proactive event scheduling and plummeting performance to 31.1% and 13.6%.

Comprehensive in-depth analyses of these ablation modes, along with real-time inference speed profiling, are deferred to Appendix C.2.

5.3 Real-World Robot Evaluation

To evaluate EventVLA in physical environments, we deploy our framework on the ARX ACONE bimanual robot across four non-Markovian manipulation tasks, each tested over 20 independent trials. These tasks explicitly evaluate diverse cognitive memory capabilities under real-world settings: 1) ***Find Block Easy*** and ***Find Block Hard*** require the model to remember the spatial location of a hidden block after only transient visual exposure. 2) ***Pick-X-Times*** tests counting logic, requiring the robot to read a randomized number and manipulate a block accordingly. 3) ***Pick in Order*** evaluates in-context memory by asking the robot to reproduce a randomized sequence initially pointed out by a stick. We benchmark EventVLA against a state-of-the-art non-memory model, $\pi_{0.5}$ [1], and a reproduced memory-augmented baseline, π_{MEM} [9].

As illustrated in Fig. 4, the purely reactive $\pi_{0.5}$ policy almost entirely fails across all tasks (achieving only 0% to 10% success rates) as it fundamentally lacks the historical context required to infer occluded states. While the memory-augmented π_{MEM} baseline demonstrates partial improvements (e.g., 50% on *Find Block Easy*), its performance degrades significantly on more complex multi event-requiring tasks like *Pick-X-Times* (30%) and *Pick in Order* (40%) due to the lossy compression of long-term history. In stark contrast, EventVLA achieves commanding success rates of 90%, 60%, 90%, and 75% across the four tasks, respectively. This robust physical performance validates that the KEM module can effectively extract and retain critical transient visual cues, empowering the VLA model with long-horizon memory awareness in the real world.

6 Limitations

While EventVLA effectively captures transient visual evidence, its bounded event buffer limits scalability in exceptionally long-horizon tasks (e.g., $>$ 10 minutes) with high event densities. Such scenarios risk buffer saturation and premature eviction of early historical cues. Future work will explore hierarchical memory or compressed representations to manage massive event sequences.

7 Conclusion

We introduced EventVLA, an end-to-end framework tackling non-Markovian long-horizon manipulation via sparse visual evidence memory. By uniting rule-based visual anchors with a foresight-driven Keyframe Evidence Memory (KEM) module, EventVLA proactively captures task-critical transient events, completely avoiding the redundancy of dense memory. Furthermore, we proposed RoboTwin-MeM, a diagnostic benchmark for evaluating intermediate memory capabilities. Extensive evaluations across 17 simulation and 4 real-world tasks demonstrate that EventVLA significantly outperforms state-of-the-art memory-augmented VLAs, ensuring robust memory-requiring long-horizon physical execution.

Acknowledgments

This work is supported by Shanghai Artificial Intelligence Laboratory.

References

[1] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[2] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, et al. $\pi^*_{0.6}$: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[4] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. *arXiv preprint arXiv:2403.03954*, 2024.

[5] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[6] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026.

[7] A. Sridhar, J. Pan, S. Sharma, and C. Finn. Memer: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.

[8] T. Chen, Y. Wang, M. Li, Y. Qin, H. Shi, Z. Li, Y. Hu, Y. Zhang, K. Wang, Y. Chen, et al. Rmbench: Memory-dependent robotic manipulation benchmark with insights into policy design. *arXiv preprint arXiv:2603.01229*, 2026.

[9] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, et al. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.

[10] L. Xiao, J. Li, J. Gao, F. Ye, Y. Jin, J. Qian, J. Zhang, Y. Wu, and X. Yu. Ava-vla: Improving vision-language-action models with active visual attention. *arXiv preprint arXiv:2511.18960*, 2025.

[11] A. Bulatov, Y. Kuratov, and M. Burtsev. Recurrent memory transformer. *Advances in Neural Information Processing Systems*, 35:11079–11091, 2022.

[12] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.

[13] H. Li, S. Yang, Y. Chen, Y. Tian, X. Yang, X. Chen, H. Wang, T. Wang, F. Zhao, D. Lin, et al. Cronusvla: Transferring latent motion across time for multi-frame prediction in manipulation. *arXiv e-prints*, pages arXiv–2506, 2025.

[14] X. Wang, X. Gao, J. Fu, Z. Li, D. Fortier, G. Mullins, A. Kolobov, and B. Guo. Lola: Long horizon latent action learning for general robot manipulation. *arXiv preprint arXiv:2512.20166*, 2025.

[15] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[16] S. Liu, B. Li, K. Ma, L. Wu, H. Tan, X. Ouyang, H. Su, and J. Zhu. Rdt2: Exploring the scaling limit of umi data towards zero-shot cross-embodiment generalization. *arXiv preprint arXiv:2602.03310*, 2026.

[17] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.

[18] T. Chen, Y. Mu, Z. Liang, Z. Chen, S. Peng, Q. Chen, M. Xu, R. Hu, H. Zhang, X. Li, et al. G3flow: Generative 3d semantic flow for pose-aware and generalizable object manipulation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1735–1744, 2025.

[19] J. Wen, Y. Zhu, J. Li, Z. Tang, C. Shen, and F. Feng. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025.

[20] M. Lin, P. Ding, S. Wang, Z. Zhuang, Y. Liu, X. Tong, W. Song, S. Lyu, S. Huang, and D. Wang. Hif-vla: Hindsight, insight and foresight through motion representation for vision-language-action models. *arXiv preprint arXiv:2512.09928*, 2025.

[21] Z. Liang, Y. Li, T. Yang, C. Wu, S. Mao, T. Nian, L. Pei, S. Zhou, X. Yang, J. Pang, et al. Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies. *arXiv preprint arXiv:2508.20072*, 2025.

[22] G. Yang, T. Zhang, H. Hao, W. Wang, Y. Liu, D. Wang, G. Chen, Z. Cai, J. Chen, W. Su, et al. Vlaser: Vision-language-action model with synergistic embodied reasoning. *arXiv preprint arXiv:2510.11027*, 2025.

[23] W. Shen, Y. Liu, Y. Wu, Z. Liang, S. Gu, D. Wang, T. Nian, L. Xu, Y. Qin, J. Pang, et al. Expertise need not monopolize: Action-specialized mixture of experts for vision-language-action learning. *arXiv preprint arXiv:2510.14300*, 2025.

[24] J. Wen, Y. Zhu, M. Zhu, Z. Tang, J. Li, Z. Zhou, X. Liu, C. Shen, Y. Peng, and F. Feng. Diffusionvla: Scaling robot foundation models via unified diffusion and autoregression. In *Forty-second International Conference on Machine Learning*, 2025.

[25] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.

[26] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, volume 2025, pages 54277–54296, 2025.

[27] H. Tan, P. Co, Y. Xu, S. Rong, Y. Ji, C. Chi, X. Chen, Q. Zhang, Z. Zhao, P. Wang, et al. Actionsketcher: From reasoning to action via visual sketches for long-horizon robotic manipulation. *arXiv preprint arXiv:2601.01618*, 2026.

[28] H. Wang, Z. Jing, J. Ao, S. Song, X. Li, G. Huang, and C. Bai. Beyond short-horizon: Vq-memory for robust long-horizon manipulation in non-markovian simulation benchmarks. *arXiv preprint arXiv:2603.09513*, 2026.

[29] M. Torne, A. Tang, Y. Liu, and C. Finn. Learning long-context diffusion policies via past-token prediction. *arXiv preprint arXiv:2505.09561*, 2025.

[30] Y.-L. Wei, H. Liao, Y. Lin, P. Wang, Z. Liang, G. Liu, and W.-S. Zheng. Cyclemanip: Enabling cyclic task manipulation via effective historical perception and understanding. *arXiv preprint arXiv:2512.01022*, 2025.

[31] H. Jang, S. Yu, H. Kwon, H. Jeon, Y. Seo, and J. Shin. Contextvla: Vision-language-action model with amortized multi-frame context. *arXiv preprint arXiv:2510.04246*, 2025.

[32] M. Lin, X. Liang, B. Lin, L. Jingzhi, Z. Jiao, K. Li, Y. Ma, Y. Liu, S. Zhao, Y. Zhuang, et al. Echovla: Robotic vision-language-action model with synergistic declarative memory for mobile manipulation. *arXiv preprint arXiv:2511.18112*, 2025.

[33] Y. Lei, Z. Liang, H. Zhang, and P. Luo. Vpwem: Non-markovian visuomotor policy with working and episodic memory. *arXiv preprint arXiv:2603.04910*, 2026.

[34] L. Tan, J. Li, and G. Jing. Memoact: Atkinson-shiffrin-inspired memory-augmented visuomotor policy for robotic manipulation. *arXiv preprint arXiv:2603.18494*, 2026.

[35] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[36] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.

[37] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T.-k. Chan, et al. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2024.

[38] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martín-Martín, C. Wang, G. Levine, M. Lingelbach, J. Sun, et al. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93. PMLR, 2023.

[39] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[40] H. Fang, M. Grotz, W. Pumacay, Y. R. Wang, D. Fox, R. Krishna, and J. Duan. Sam2act: Integrating visual foundation model with a memory architecture for robotic manipulation. *arXiv preprint arXiv:2501.18564*, 2025.

[41] E. Cherepanov, N. Kachaev, A. K. Kovalev, and A. I. Panov. Memory, benchmark & robots: A benchmark for solving complex tasks with reinforcement learning. *arXiv preprint arXiv:2502.10550*, 2025.

[42] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36: 44776–44791, 2023.

[43] S. Han, B. Qiu, Y. Liao, S. Huang, C. Gao, S. Yan, and S. Liu. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. *Advances in Neural Information Processing Systems*, 38, 2026.

[44] H. Lei, W. Song, H. Zhang, J. Pei, J. Chen, H. Yan, H. Zhao, P. Ding, Z. Zhang, L. Huang, et al. Robomemarena: A comprehensive and challenging robotic memory benchmark. *arXiv preprint arXiv:2605.10921*, 2026.

[45] F. Xiang, Y. Qin, K. Mo, Y. Xia, H. Zhu, F. Liu, M. Liu, H. Jiang, Y. Yuan, H. Wang, et al. Sapien: A simulated part-based interactive environment. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11097–11107, 2020.

[46] S. Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.

[47] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[48] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Appendix

A Implementation Details of EventVLA

A.1 Training Formulations and Curriculum Strategy

To obtain ground-truth (GT) keyframe supervisions, we leverage an offline automated labeling pipeline powered by Qwen3-VL [15]. By parsing raw demonstration videos alongside task descriptions, the VLM extracts the exact timestamps of task-critical intermediate events, denoted as t^*. However, in physical robot execution, keyframe semantics inherently exhibit temporal ambiguity—frames immediately preceding or succeeding t^* are often equally valid for capturing the visual evidence. To prevent noisy gradients caused by rigid binary supervision, we smooth the annotations into a soft target vector $\mathbf{y}_t \in [0,1]^H$ utilizing a raised cosine kernel. Specifically, for a future step i within a dilation radius R of a GT event t^*, the soft target is defined as $y_t^i = 0.5(1 + \cos(\pi \frac{|t+i-t^*|}{R}))$.

To supervise the chunk-wise keyframe predictions against these temporally smoothed annotations, we formulate the Keyframe Evidence Memory loss L_{kem} as a sequence-averaged Binary Cross-Entropy (BCE) objective. This explicitly aligns each predicted scalar probability $\hat{p}_t^i \in [0,1]$ with its corresponding soft target $y_t^i \in [0,1]$ across the entire future action horizon H:

$$L_{\text{kem}} = -\frac{1}{H}\sum_{i=1}^{H}\left[y_t^i \log(\hat{p}_t^i) + (1-y_t^i)\log(1-\hat{p}_t^i)\right] \tag{6}$$

The entire framework is then optimized end-to-end via a joint objective that couples memory awareness with precise motor control:

$$L = L_{\text{action}} + \lambda L_{\text{kem}} \tag{7}$$

where L_{action} denotes the standard continuous action generation loss (e.g., regression or flow-matching), and λ serves as a balancing coefficient to appropriately scale the memory supervision.

During training, constructing the event buffer E_t dynamically from the model's own predictions in the early stages causes severe training instability, whereas relying exclusively on GT keyframes introduces a critical train-test distribution shift since GT keyframes are unavailable at test time. To bridge this gap, we implement a scheduled teacher-to-student curriculum. We introduce an annealing parameter α that linearly decays from 1 to 0 over the training duration. At each step, the framework decides whether to commit an observation to E_t using the GT keyframes with probability α (teacher-forcing), or relying on its own thresholded predictions ($\hat{p}_t^i \geq \tau_{\text{commit}}$) with probability $1-\alpha$. This gradual transition ensures stable initial convergence while forcing the VLA policy to eventually adapt to its own autonomous memory updating cadence.

A.2 Online Inference and Post-Processing

During online inference, the chunk-wise prediction $\hat{\mathbf{p}}_t$ naturally yields clustered, temporally continuous high-probability scores around an unfolding semantic event. To prevent redundant frames of the same visual event from flooding the bounded buffer E_t, we compress the H-dimensional probability vector into discrete, sparse write events via a rigorous post-processing extraction pipeline.

First, we identify a set of local probability peaks $\mathcal{K}_t$ by applying the confidence threshold τ_{commit} coupled with a 1D Non-Maximum Suppression (NMS) algorithm. Specifically, a future step index i is selected as a candidate peak if its probability exceeds the threshold and represents the local maximum within a sliding temporal window of radius w:

$$\mathcal{K}_t = \left\{ i \in \{1,\dots,H\} \;\middle|\; \hat{p}_t^i \geq \tau_{\text{commit}} \wedge \hat{p}_t^i = \max_{j \in \mathcal{N}_w(i)} \hat{p}_t^j \right\} \tag{8}$$

where $\mathcal{N}_w(i) = [\max(1, i-w), \min(H, i+w)]$ denotes the NMS neighborhood.

While NMS effectively isolates local peaks, rapid consecutive events might still trigger excessive memory writes. To strictly enforce operational sparsity, a temporal cooldown period C is evaluated

sequentially over the candidates. A candidate peak $i \in \mathcal{K}_t$ *is officially validated and committed to* E_t if and only if $(t+i) - t_{\text{last}} > C$, where t_{last} denotes the absolute physical timestamp of the most recently committed keyframe. Through this cascading extraction mechanism, the framework mathematically distills the dense predictive landscape into an optimal, highly sparse subset. This guarantees that memory allocation remains strictly tied to novel interactive evidence, maximizing information retention while seamlessly adhering to real-time execution constraints and bounded memory buffer size $N_{\max}$.

A.3 Automated Keyframe Annotation Pipeline

To circumvent the prohibitive costs associated with dense manual frame annotation for long-horizon tasks, we develop an automated, highly scalable keyframe labeling pipeline powered by Large Vision-Language Models (VLMs). Specifically, we deploy the state-of-the-art **Qwen3-VL-235B-A22B-Instruct-FP8** [15] model on a local server equipped with 8 NVIDIA A800 GPUs using the vLLM [48]s framework.

Data Pre-processing and In-Context Learning. Rather than feeding an unmanageably long continuous video stream directly into the VLM, we uniformly sample the temporal horizon into a discrete set of frames (e.g., 128 frames per episode). To ensure robust spatial awareness, particularly in scenarios involving severe occlusions, we extract and concatenate multi-view observations (e.g., global head camera and wrist camera) for each sampled timestep. Crucially, to align the VLM's outputs with our specific definition of transient visual evidence, we employ an In-Context Learning (ICL) strategy. The prompt includes a few-shot demonstration from identical or similar tasks, containing the sampled frames alongside their ground-truth keyframe steps, establishing a rigorous template for temporal alignment and JSON-formatted outputs.

The exact system prompt utilized for the automated pipeline is presented below:

System Prompt for VLM-based Keyframe Annotation

System Prompt:
You are an expert robot-video keyframe annotator.
CRITICAL REQUIREMENT:
- The annotation MUST strictly follow `task_instruction`.
- Treat `task_instruction` as the primary objective definition; if visuals are ambiguous, prioritize consistency with it.
Episode metadata:
- `episode_id`: <episode_id>
- `task_instruction`: <task_instruction>
- `total_frames`: <total_frames>
- `required_keyframe_count`: <num_keyframes>
- `provided_views`: <selected_views>
What to annotate:
1. Find key state transitions for this task (e.g., stable grasp acquired, object placed, cycle transition).
2. Keep keyframes representative and temporally ordered across the full task progress.
3. For repeated pick/place cycles, pick the most stable and recognizable moments per cycle.
Output format constraints:
1. Return JSON only. No markdown, no explanations.
2. Format: { "keyframe_steps": [int, int, ...] }
3. `keyframe_steps` must:
    - have length exactly `<num_keyframes>`
    - be strictly increasing
    - be in [0, `<total_frames - 1>`]
    - contain no duplicates

Annotation Reliability and Error Analysis. To rigorously validate the reliability of this automated pipeline, we conducted a comprehensive cross-validation study. In the simulation environments, we compared the keyframes automatically annotated by the Qwen3-VL-235B model against the precise algorithmic ground-truth (GT) states acquired directly from the RoboTwin 2.0 [35] physics engine. The results demonstrate that the VLM's predictions exhibit an average absolute temporal error of **less than 10 timesteps**. Furthermore, when deployed on the four complex real-world bimanual tasks, the prediction error remained **within 50 timesteps** compared to human-annotated ground truth. Given that our evaluation episodes feature extremely long horizons (often exceeding 1500 to 2000 steps), this negligible temporal variance, which is naturally accommodated by our temporally smoothed soft

labels, strongly confirms that our VLM-powered automated annotation pipeline is highly reliable, precise, and ready for scalable deployment.

B Experimental Setups and Benchmarks

B.1 RoboTwin-MeM Benchmark Details

Table 4: Detailed statistics of our proposed RoboTwin-MeM Benchmark.

Task Name	Episodes	Avg. #Steps	Intermediate Keyframes	Task Instruction
Press Button Keyframe	50	430	[2,5]	Read the two number cards, press the left button as many times as the left card shows, press the middle button as many times as the right card shows, then press the right button once.
Pick the Unhidden Block	50	699	3	Open the covers one by one to identify the hidden colors, close them after inspection, then pick up the visible block whose color is not hidden.
Rearrange Blocks Hard	50	879	1	Move a chosen block from its mat to the center and press the button, return the same block to its mat and press again, then move the other block to the center and press once more.
Pick Objects in Order	50	1124	3	Open the covers one by one to observe the objects inside, close them after inspection, then pick up the objects in the observed order.
Find Seal and Seal Stamp	50	1338	[1,4]	Open the covers one by one to find and take out the seal, close the cover after inspection, stamp with it, then return it under its original cover.
Reproduce Route	50	1417	4	Move the center red block to the four blue pads in a random order, returning it to the center. Then use the outside red block to repeat the same pad order.
Put Back Block Hard	50	1468	2	For each row, move the center block to a randomly selected outer pad in the same row, move the arm back, return the block to the center, move the arm back again, and press the button. Finally, move both blocks back to the same outer pads they first visited, then press the button.
Cover Blocks Hard	50	1544	4	Open the covers one by one, close them after inspection, then reopen them in the order: red, green, blue, yellow.

As introduced in Sec. 4 of the main text, RoboTwin-MeM is a diagnostic simulation suite specifically engineered to isolate and evaluate genuinely non-Markovian robotic manipulation. Unlike conventional long-horizon environments where the workspace state remains persistently visible, RoboTwin-MeM enforces strict visual occlusions and temporal delays. In these tasks, critical visual evidence, such as the hidden color of a block, the identity of an object under a cover, or a randomly generated spatial sequence, manifests only transiently during intermediate interactions before becoming completely unobservable.

To systematically quantify memory capacity, RoboTwin-MeM spans 8 complex bimanual manipulation tasks with exceptionally long execution horizons, ranging from an average of 430 to 1,544 steps per episode. The difficulty of each task is explicitly parameterized by $n \in [1, 5]$, which defines the

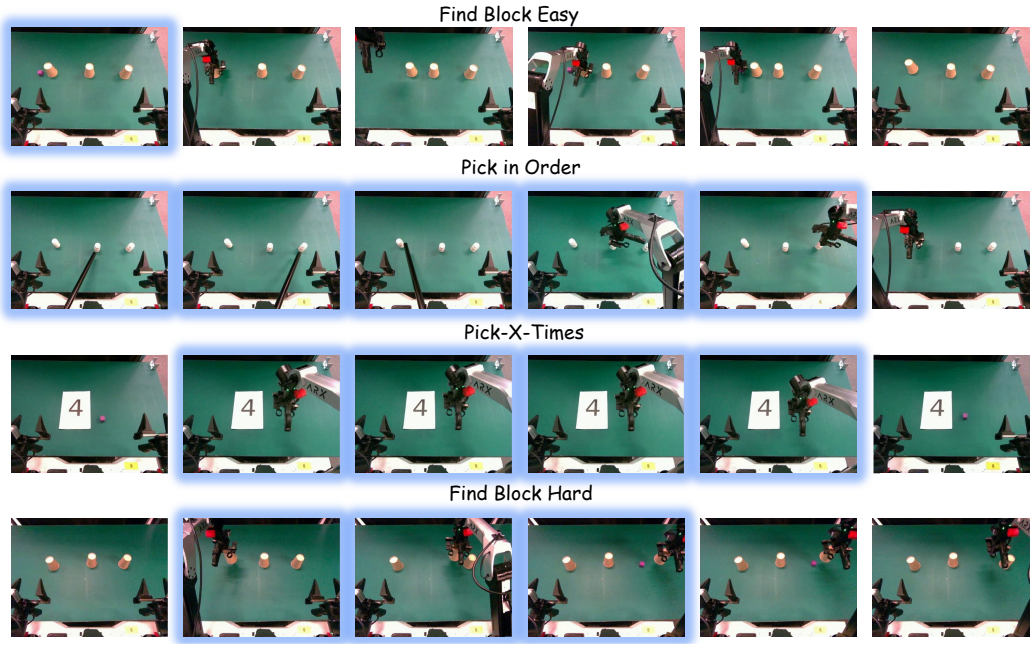


Figure 5: Expanded real-world execution sequences of EventVLA across the four manipulation tasks. The specific task-critical intermediate keyframes, which the policy autonomously captures and commits to memory, are highlighted with blue borders.

exact number of intermediate keyframe events the robot must autonomously capture and retain to successfully complete the instruction.

For instance, memory-intensive tasks like *Cover Blocks Hard* ($n = 4$) and *Pick Objects in Order* ($n = 3$) require the robot to lift opaque covers to inspect hidden attributes, remember them after the covers are closed, and execute subsequent pick-and-place actions based on that stored memory. Similarly, *Press Button Keyframe* requires the robot to read randomized number cards and translate them into a sequential counting and pressing logic. As visualized in Fig. 3, these transient, interaction-driven keyframes (highlighted with blue borders) serve as the critical informational bridge between past observations and future actions.

The comprehensive task statistics, including the average number of steps, the required intermediate keyframe count n, and the specific language instructions for all 8 evaluation tasks, are detailed in Table 4.

B.2 Real-world Tasks Details

To supplement the single-frame task overviews provided in Fig. 4, Fig. 5 presents the expanded, step-by-step temporal sequences for the four real-world manipulation tasks. These full execution rollouts illustrate exactly when transient visual evidence emerges during physical interaction. The task-critical intermediate keyframes that the policy must autonomously capture and commit to its dynamic event buffer, such as briefly exposing a hidden block, reading a randomized number, or observing a specific sequence pointed out by a stick, are explicitly highlighted with blue borders. By successfully isolating and retaining these sparse states, EventVLA effectively bridges the temporal gap required for non-Markovian control.

B.3 Network Architecture and Hyper-parameters

For RMBench and RoboTwin-MeM, our EventVLA framework is built upon the open-source QwenOFT [46] architecture, which utilizes Qwen3-VL-4B-Instruct as the foundational Vision-Language Model (VLM). The visual observations are resized to 224×224 before being processed by the vision encoder.

During the training phase, the entire framework is optimized end-to-end using the AdamW optimizer for 80,000 training steps. We apply a differential learning rate strategy to ensure stable convergence: the pre-trained VLM backbone is fine-tuned with a lower learning rate of 1×10^{-5}, while the newly initialized components (the action head and the Keyframe Evidence Memory prediction head) are trained with a higher learning rate of 1×10^{-4}. The action prediction horizon (H) is set to 50 steps for all tasks.

To explicitly reflect the different memory demands of our evaluated benchmarks, we configure the memory modules differently. For **RMBench**, which primarily evaluates foundational visual anchoring without the need for intermediate transient memory, the policy is trained exclusively with initial and short-term visual anchors. The detailed network architecture and training hyper-parameters for RMBench are summarized in Table 5.

Table 5: Network Architecture and Training Hyper-parameters for RMBench.

Configurations	**Values**
Network Architecture	
Base VLM	Qwen3-VL-4B-Instruct
Action Model Type	Optimized Fine-Tuning (OFT)
Action Dimension	14
Action Horizon (H)	50
Image Resolution	224×224
Training Hyper-parameters	
Optimizer	AdamW
Training Steps	80,000
Base VLM Learning Rate	1×10^{-5}
Action Head Learning Rate	1×10^{-4}
Per-Device Batch Size	4
Gradient Accumulation Steps	1
Memory Module Status	Visual Anchors Only
Visual Anchors A_t	o_0, o_{t-30}, o_{t-15}

Conversely, for the strictly non-Markovian **RoboTwin-MeM**, the full Keyframe Evidence Memory (KEM) module is activated. To ensure early training stability and bridge the train-test distribution shift, we apply a scheduled teacher-to-student curriculum, where the teacher-forcing probability α decays linearly from 1.0 to 0.0 over the training duration. As detailed in Table 6, we also introduce specific hyper-parameters to govern the online memory extraction pipeline. The chunk-wise keyframe predictions are filtered using a commit confidence threshold of $\tau_{\text{commit}} = 0.55$. To enforce rigorous memory sparsity, we apply a 1D Non-Maximum Suppression (NMS) sliding window with a radius of $w = 8$, followed by a temporal cooldown period of $C = 10$ steps between consecutive memory writes. Finally, the dynamic event buffer is bounded by a maximum capacity of $N_{\text{max}} = 5$, managed by a FIFO eviction policy to satisfy real-time computational constraints.

For physical deployment on the real-world robot platform, we adapt our framework utilize $\pi_{0.5}$ [1] as the foundational Vision-Language-Action Model. The action head is configured to predict a 32-dimensional continuous action over a horizon of $H = 50$ steps. During fine-tuning on real-world demonstrations, the entire framework is jointly optimized for 60,000 steps using the AdamW optimizer with a global batch size of 32 in bfloat16 precision. We apply a uniform peak learning rate of 5×10^{-5} for both the base VLM and the newly initialized heads, following a cosine decay schedule with 2,000 warm-up steps. To manage the Keyframe Evidence Memory (KEM) module during physical execution, we maintain a commit confidence threshold of $\tau_{\text{commit}} = 0.55$, a maximum event buffer capacity of $N_{\text{max}} = 5$, an NMS temporal window radius of $w = 8$, and a commit cooldown period of $C = 10$. The keyframe loss weight λ is set to 0.1, alongside a scheduled teacher-to-student curriculum where α decays linearly from 1.0 to 0.0. The comprehensive network architecture and training details for the real-world tasks are summarized in Table 7.

Table 6: Network Architecture and KEM Hyper-parameters for RoboTwin-MeM.

Configurations	**Values**
Network Architecture & Basic Training	
Base VLM	Qwen3-VL-4B-Instruct
Action Model Type	Optimized Fine-Tuning (OFT)
Action Horizon (H)	50
Optimizer	AdamW
Training Steps	80,000
Base VLM Learning Rate	1×10^{-5}
KEM & Action Head Learning Rate	1×10^{-4}
Per-Device Batch Size	4
Gradient Accumulation Steps	1
Keyframe Evidence Memory (KEM) Settings	
Memory Module Status	Full EventVLA (VA + KEM)
Visual Anchors A_t	o_0, o_{t-30}, o_{t-15}
Teacher-Forcing Annealing (α)	Linear decay ($1.0 \rightarrow 0.0$)
Commit Confidence Threshold (τ_{commit})	0.55
Max Event Buffer Size (N_{max})	5
NMS Temporal Window Radius (w)	8
Commit Cooldown Period (C)	10
Keyframe Loss Weight (λ)	0.1

Table 7: Network Architecture and KEM Hyper-parameters for real-robot.

Configurations	**Values**
Network Architecture & Basic Training	
Base VLM	PaliGemma ($\pi_{0.5}$)
Image Resolution	224×224
Text Sequence Length	200
Action Horizon (H)	50
Action Dimension	32
Optimizer	AdamW
Optimizer Hyper-parameters	$\beta_1 = 0.9$, $\beta_2 = 0.95$, $eps = 1e-8$
Weight Decay	0.01
Training Steps	60,000
Warm-up Steps	2,000
Base VLM Learning Rate	5×10^{-5}
KEM & Action Head Learning Rate	5×10^{-5}
Minimum Learning Rate	5×10^{-6}
Learning Rate Schedule	cosine decay with minimum LR
Global Batch Size	32
Numerical Precision	bfloat16
Keyframe Evidence Memory (KEM) Settings	
Memory Module Status	Full EventVLA (VA + KEM)
Visual Anchors A_t	$o_0, o_{t-60}, o_{t-40}, o_{t-20}$
Teacher-Forcing Annealing (α)	lienar decay ($1.0 \rightarrow 0.0$)
Commit Confidence Threshold (τ_{commit})	0.55
Max Event Buffer Size (N_{max})	5
NMS Temporal Window Radius (w)	8
Commit Cooldown Period (C)	10
Keyframe Loss Weight (λ)	0.1

C Extended Experimental Results and Analysis

Table 8: RMBench benchmark results. (**Bold** : best; Underlined: second-best).

Tasks	Observe and Pick Up	Rearrange Blocks	Put Back Block	Swap Blocks	Swap T	Battery Try	Blocks Ranking Try	Cover Blocks	Press Button	*Total average*
▼ *Non Memory-based Vision-language-action Models:*										
DP	1%	0%	0%	11%	20%	10%	10%	0%	0%	5.8%
ACT	1%	29%	0%	2%	2%	19%	0%	0%	0%	5.9%
$\pi_{0.5}$	9%	13%	11%	24%	15%	16%	6%	0%	0%	10.4%
X-VLA	9%	13%	18%	16%	3%	26%	1%	2%	0%	9.8%
QwenOFT	0%	0%	0%	0%	0%	14%	37%	0%	0%	5.6%
▼ *Dual-system Memory-based Vision-language-action Models:*										
MemER	7%	17%	0%	14%	7%	27%	0%	6%	0%	8.7%
Mem-0	4%	89%	90%	67%	14%	28%	18%	68%	0%	42.0%
▼ *End-to-end Memory-based Vision-language-action Models:*										
MemoryVLA (OpenVLA)	0%	22%	50%	17%	9%	25%	12%	40%	0%	19.4%
MemoryVLA (QwenOFT)	2%	53%	81%	76%	9%	33%	53%	69%	0%	41.7%
EventVLA (w/o initial)	10%	64%	63%	16%	8%	**39%**	**87%**	15%	2%	33.7%
EventVLA (w/o short-term)	15%	34%	20%	18%	**94%**	16%	14%	4%	0%	23.8%
EventVLA (visual anchors only)	**21%**	**96%**	**95%**	**96%**	87%	35%	81%	**97%**	**3%**	**67.8%**

C.1 Detailed Per-Task Breakdown on RMBench

Due to space limits in the main text, we present the comprehensive task-level breakdown for all baseline and ablation models on the RMBench suite in Table 8, which is the task-specific expanded version of Table 1.

The detailed breakdown clearly illustrates that our streamlined configuration, EventVLA (visual anchors only), significantly outperforms both non-memory and prior memory-augmented baselines across the vast majority of tasks. Notably, on memory-intensive structural manipulation tasks such as *Rearrange Blocks* (96%), *Put Back Block* (95%), *Swap Blocks* (96%), and *Cover Blocks* (97%), EventVLA achieves near-perfect success rates. This demonstrates that our rule-based visual anchoring mechanism effectively captures the persistent spatial layouts and fixed motion styles required for conventional memory-oriented scenarios without the need for complex state compression. Furthermore, the ablation variants (w/o initial and w/o short-term) show severe performance degradation across almost all tasks, confirming that both the initial global spatial reference and short-term motion cues are indispensable components of the visual anchors.

C.2 Extended Ablation Analysis and Inference Efficiency

Extended Ablation Analysis. To deeply understand the contributions of individual design choices within the Keyframe Evidence Memory (KEM) module, we expand upon the ablation studies conducted on the RoboTwin-MeM suite (summarized in the main text Sec. 5.2 and Table 2).

- **Memory Representation (Explicit Images vs. Implicit Bank):** In the implicit memory bank variant, captured keyframes are aggregated into a compressed latent embedding rather than appended as explicit raw images. When handling complex tasks that demand the retention of multiple distinct events (e.g., $n \geq 3$), squeezing disparate historical features into a single latent vector creates a severe information bottleneck. Explicit raw image concatenation avoids this lossy compression, providing complete, lossless contextual evidence for the VLA's multi-frame attention mechanism.
- **Supervision Strategy (Soft Labels vs. Hard Labels):** Physical keyframe events naturally span continuous temporal windows. Replacing our raised cosine soft labels with strict binary targets induces extreme label sparsity and heavily penalizes valid adjacent frames. This rigid supervision destabilizes the predictive head, ultimately causing it to fail in triggering essential memory writes. Soft labels provide the necessary temporal tolerance for robust event capture in environments with execution variance.
- **Buffer Management (The Necessity of NMS and Capacity):** Without the 1D Non-Maximum Suppression (NMS) post-processing algorithm, redundant adjacent frames

Table 9: Ablation Study on EventVLA's Inference Speed. *Latency* denotes the average time in seconds required for generating each chunk (s/chunk), while *Throughput* denotes the average number of chunks generated per second (chunks/s).

Tasks		n=1	n=2	n=3		n=4			n=5	*Total average*
		Rearrange Blocks Hard	Put Back Block Hard	Pick Objects in Order	Pick the Unhidden Block	Cover Blocks Hard	Find Seal Stamp	Reproduce Route	Press Button Keyframe	
QwenOFT	*Latency*	0.31	0.36	0.36	0.39	0.41	0.39	0.30	0.32	0.36
	Throughput	3.21	2.82	2.82	2.56	2.57	2.62	3.46	3.20	2.91
EventVLA (visual anchors only)	*Latency*	0.92	0.78	1.08	0.83	1.05	1.02	0.95	1.08	0.96
	Throughput	1.11	1.35	0.93	1.21	0.96	1.02	1.07	0.94	1.07
EventVLA (VA+KEM)	*Latency*	0.90	0.88	1.20	0.97	1.22	1.11	1.25	1.22	1.09
	Throughput	1.13	1.16	0.84	1.03	0.83	0.92	0.81	0.83	0.94

rapidly flood the bounded dynamic event buffer. Conversely, a strictly minimal buffer (e.g., $N_{\max} = 2$) inherently lacks the structural capacity required for complex, multi-stage tasks. Both scenarios lead to premature buffer saturation and trigger early FIFO eviction, which mistakenly discards foundational historical evidence (such as the first observed hidden color) before it can be utilized. This underscores that NMS-driven event sparsity and adequate memory capacity are both vital.

- **Foresight Horizon (The Impact of Action Chunk Size):** The execution chunk size governs KEM's look-ahead window. Shrinking this horizon truncates the model's predictive capacity, preventing the keyframe head from effectively anticipating and scheduling upcoming transient events, thus neutralizing KEM's proactive memory commitment capability.

Inference Efficiency. To verify that EventVLA can be effectively deployed on physical robots, we meticulously evaluate its real-time inference speed. Table 9 details the latency and throughput of our framework across the RoboTwin-MeM benchmark.

The purely reactive QwenOFT baseline achieves an average throughput of 2.91 Hz with a latency of 0.36 seconds. Incorporating external visual anchors slightly increases the computational footprint due to the extended multi-frame input sequence, resulting in an average throughput of 1.07 Hz. When the full EventVLA framework (incorporating dynamic KEM) is deployed, it maintains an average throughput of 0.94 Hz (1.09 seconds latency). Given that VLA policies typically operate as high-level planners alongside low-level, high-frequency controllers, this throughput comfortably meets the operational constraints for real-world robotic deployment. This confirms that our sparse memory commitment strategy strikes an optimal balance between robust non-Markovian reasoning and practical real-time execution efficiency.

D Qualitative Visualizations

D.1 Simulation Rollouts in RoboTwin-MeM

To provide an intuitive understanding of EventVLA's dynamic memory scheduling and execution process, we visualize the qualitative rollouts across all 8 strictly non-Markovian tasks in the RoboTwin-MeM benchmark. Figure 6 illustrates the successful execution sequences for four memory-intensive tasks: *Rearrange Blocks Hard*, *Pick the Unhidden Block*, *Put Back Block Hard*, and *Cover Blocks Hard*. Figure 7 demonstrates the execution pipelines for the remaining four tasks: *Press Button Keyframe*, *Pick Objects in Order*, *Find Seal and Seal Stamp*, and *Reproduce Route*.

Across these diverse scenarios, the visualizations clearly highlight how the KEM module proactively triggers sparse memory writes the exact moment transient visual evidence emerges (e.g., observing the hidden color of a block immediately after lifting an opaque cover, or reading a randomized number). By locking these critical intermediate states into the event buffer before they become unobservable, EventVLA effectively bridges the temporal gap and seamlessly guides the subsequent long-horizon manipulation.

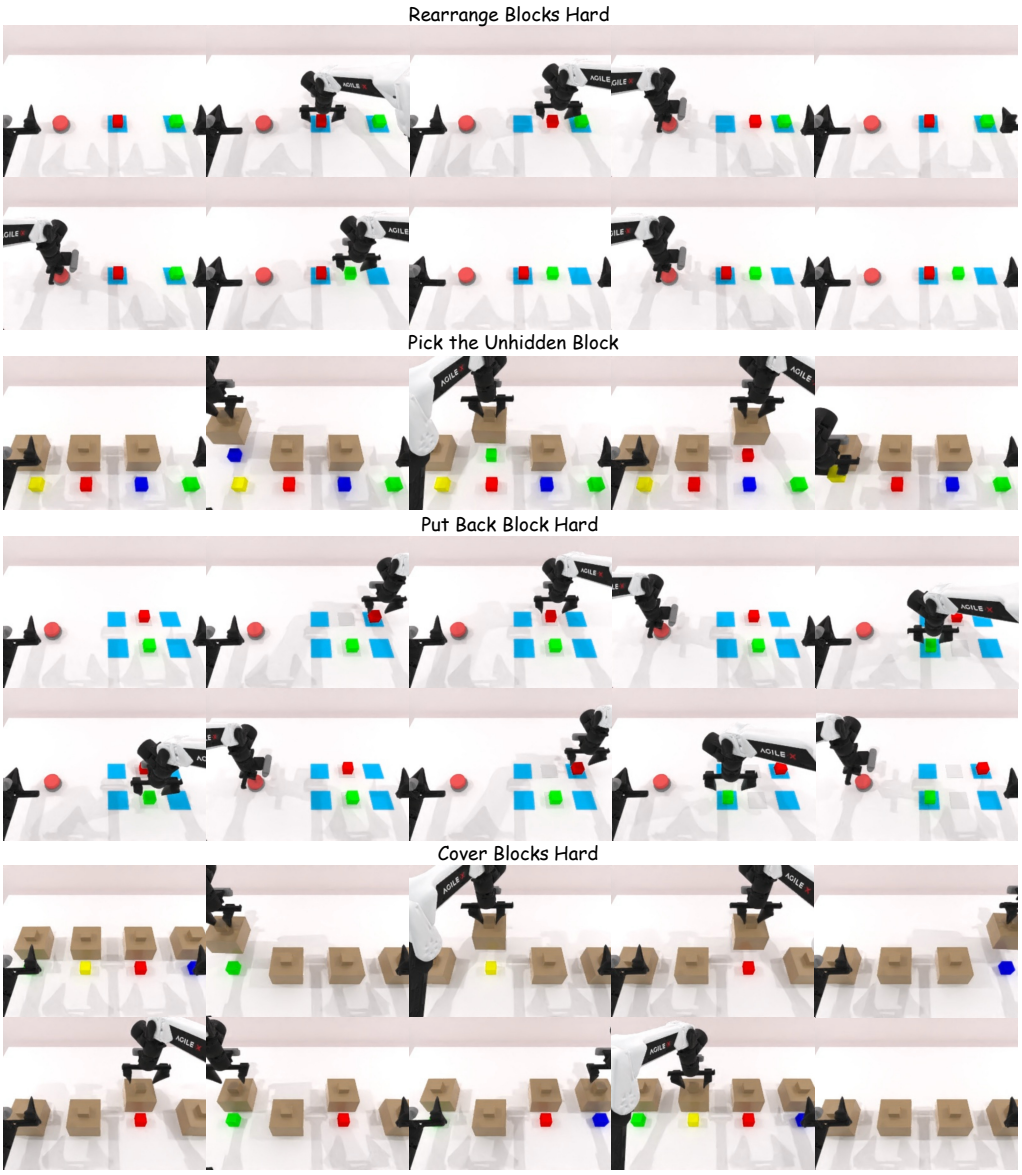


Figure 6: Qualitative rollouts of EventVLA on four RoboTwin-MeM simulation tasks: *Rearrange Blocks Hard*, *Pick the Unhidden Block*, *Put Back Block Hard*, and *Cover Blocks Hard*.

D.2 Real-World Robot Execution Sequences

To further validate the practical efficacy of our framework, we provide qualitative execution sequences of EventVLA deployed on the real-world ARX ACONE bimanual robot. Figure 8 showcases the successful completion of four memory-intensive manipulation tasks: *Find Block Easy*, *Pick-X-Times*, *Find Block Hard*, and *Pick in Order*.

The visualizations demonstrate that EventVLA can robustly capture and retain critical intermediate visual cues despite real-world occlusions and randomized spatial placements. Whether reading a randomized number from a paper to dictate counting logic, or observing a stick pointing at bottles to memorize an in-context sequence, the policy successfully utilizes its sparse visual evidence memory

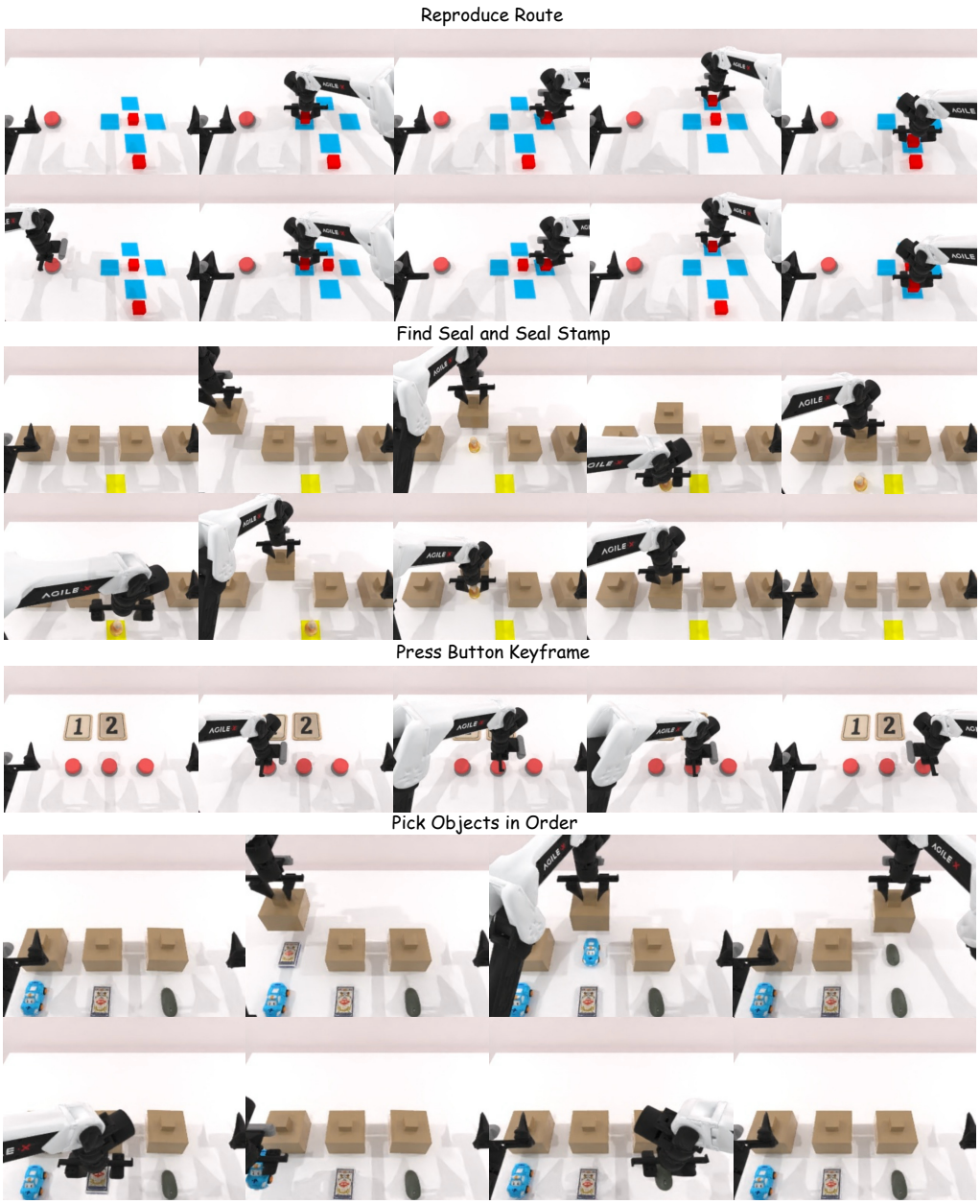
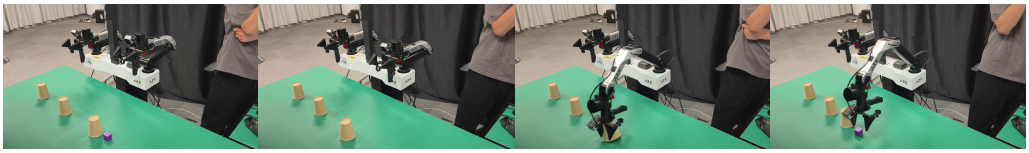


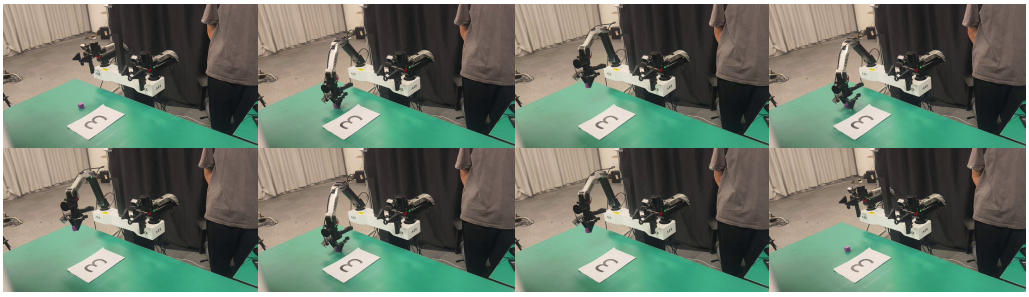
Figure 7: Qualitative rollouts of EventVLA on the remaining four RoboTwin-MeM simulation tasks: *Press Button Keyframe*, *Pick Objects in Order*, *Find Seal and Seal Stamp*, and *Reproduce Route*.

to execute complex, multi-stage physical tasks, exhibiting both strong non-Markovian remembering and spatial generalization capabilities.

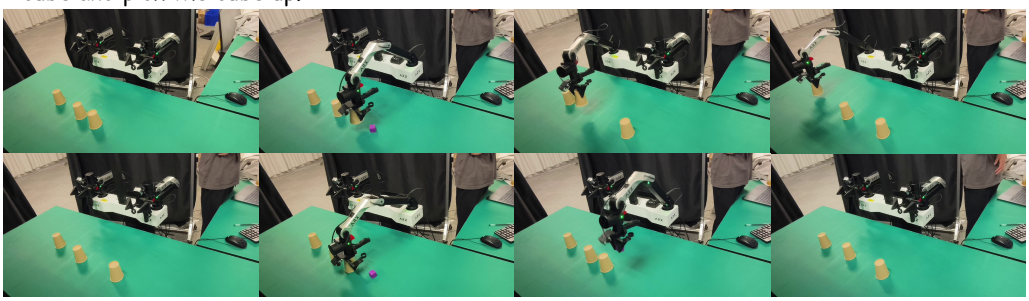
Find Block Easy: Cover the block with the nearest cup, then lift the cup covering the block and pick up the block.



Pick-X-Times: Pick up and put down the block the number of times as shown on the paper.



Find Block Hard: Lift the cups on the table one by one from left to right, checking if there is a hidden cube underneath, then put the cups down. Finally, open the cup containing the cube and pick the cube up.



Pick in Order: Pick up and put down the bottles on the table in the order which they are pointed by the stick one by one at the beginning.

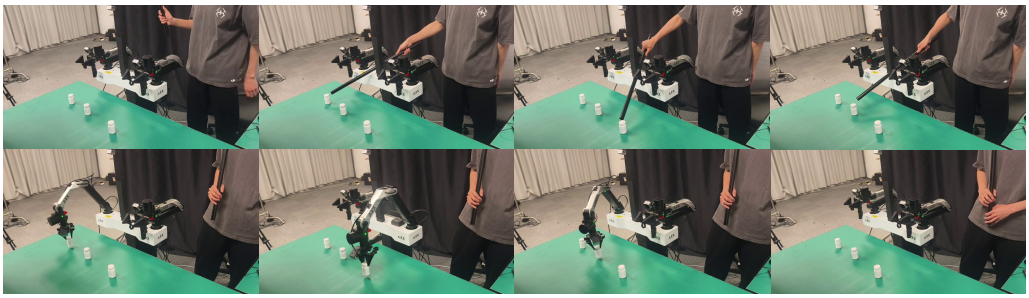


Figure 8: Qualitative real-world robot execution sequences of EventVLA on four tasks: *Find Block Easy*, *Find Block Hard*, *Pick-X-Times*, and *Pick in Order*.